Iker Acevedo Donate

CampusFP

Memoria TFG

1. De qué trata mi idea y por qué la propongo

La idea de este proyecto surge de un problema que casi todos los estudiantes y tutores hemos vivido: el caos administrativo que suponen las prácticas. Actualmente, la gestión está muy fragmentada. El alumno está en la empresa, el tutor en el centro y la comunicación entre ellos suele ser un lío de correos electrónicos que se pierden, llamadas o documentos de Excel que van y vienen. Esto hace que sea muy difícil llevar un control real de lo que el alumno hace día a día.

Mi aplicación busca centralizar todo esto en un único entorno digital. Aunque la plataforma está pensada para gestionar todo el ciclo (desde que una empresa publica una oferta hasta que se firma el convenio), **este TFG se va a centrar sobre todo en la parte del seguimiento y la comunicación**. Me parece la parte más importante porque es donde más fallos hay actualmente. Quiero que el tutor sepa de verdad cómo le va al alumno y que el alumno tenga un sitio fácil donde reportar cualquier problema sin que se quede en el olvido.

2. Objetivos del proyecto

- **Comunicación centralizada**: Crear un chat interno vinculado a la práctica para que las dudas y avisos no se mezclen con el correo personal.
- **Seguimiento real**: Que el alumno registre sus tareas y sus horas semanalmente de forma sencilla.
- **Gestión de incidencias**: Si pasa algo en la empresa, el alumno puede dar la alarma con un botón y el tutor recibe un aviso directo.
- **Supervisión del centro**: Que el centro educativo tenga una pantalla donde ver el estado de todos sus alumnos a la vez.

3. Análisis de Requisitos (Especificaciones Técnicas)

Para complementar los objetivos anteriores, he definido los requisitos técnicos que debe cumplir el sistema en esta primera fase:

3.1 Requisitos Funcionales (RF)

- **RF-01 (Gestión de Usuarios):** El sistema debe permitir el registro y login de usuarios con diferentes roles (Alumno, Tutor, Admin).
- **RF-02 (Perfil de Usuario):** El usuario debe poder visualizar sus datos de perfil una vez autenticado mediante token.
- **RF-03 (Gestión de Maestros):** Capacidad de listar Centros Educativos y Empresas colaboradoras para poblar los formularios del sistema.

3.2 Requisitos No Funcionales (RNF)

- **RNF-01 (Seguridad):** Uso de cifrado BCrypt para contraseñas y autenticación Stateless mediante JWT.
- **RNF-02 (Integridad):** Garantizar la integridad referencial de los datos mediante el motor PostgreSQL.

4. Arquitectura y Tecnologías

Para el desarrollo de este proyecto, he decidido evolucionar la idea inicial hacia una Arquitectura Cliente-Servidor (basada en API). Esta estructura separa completamente los datos del servidor de lo que el usuario ve en la pantalla.

- **Frontend (Aplicación Cliente):** He decidido utilizar el framework **Flutter** (con el lenguaje Dart). Por un lado, permite compilar el código de forma nativa para Android, iOS y escritorio con una sola base de código, facilitando que el alumno use el móvil para reportar incidencias. Por otro lado, supone un reto profesional: en mi empresa de prácticas usan herramientas de este ecosistema (FlutterFlow), y desarrollar este TFG programando directamente en Flutter me obliga a dominar el lenguaje real desde cero.

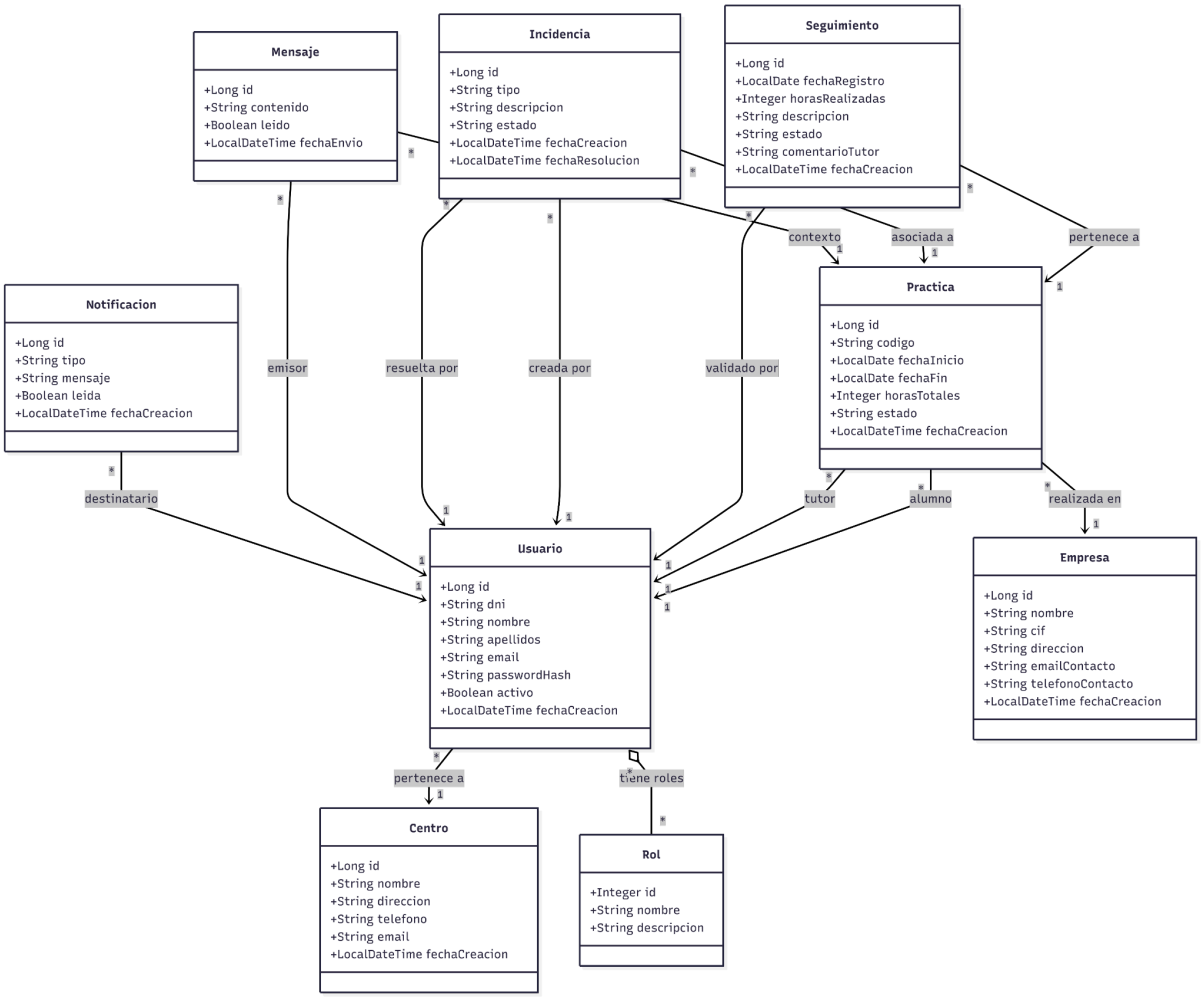
- **Backend (API REST):** He optado por **Spring Boot con Java 21**. He implementado una arquitectura de seguridad Stateless basada en **JWT (JSON Web Tokens)**, eliminando las sesiones en servidor para que el sistema sea más rápido y seguro.

- **Base de Datos:** He utilizado **PostgreSQL** (en lugar de MySQL) por su robustez en el manejo de relaciones complejas. Uso la herramienta **Flyway** para gestionar el historial de cambios de las tablas automáticamente.

4.1. Diseño de la Lógica (Backend)

Para organizar el código del servidor, he seguido un patrón orientado a objetos, separando claramente los Controladores (que reciben las peticiones de Flutter), los Servicios (donde está la lógica y las validaciones) y los Repositorios (que hablan con la base de datos).

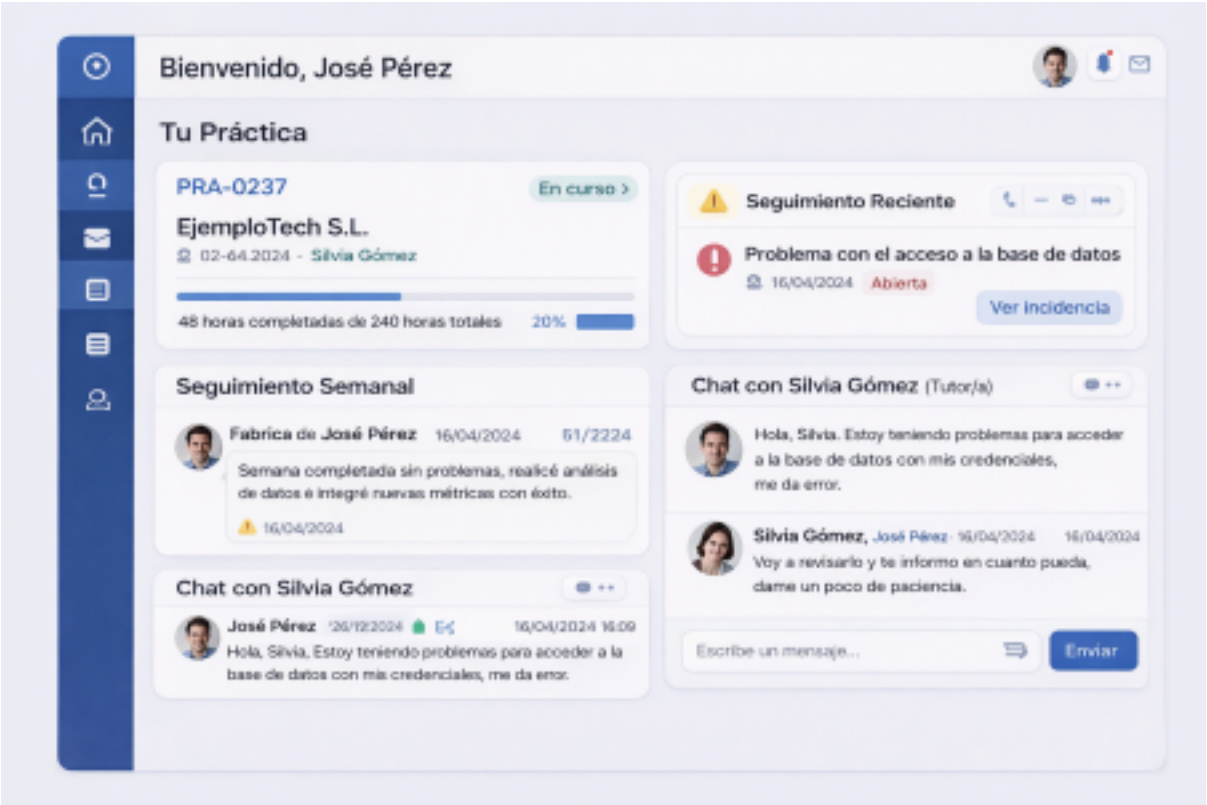
A continuación, presento el diagrama de clases (UML) del núcleo del sistema, centrado en la autenticación y la gestión de entidades maestras (Centros y Empresas), que constituye el primer gran hito de desarrollo del proyecto.



5. Diseño de la Interfaz por Roles

Aquí es donde se ve cómo funciona la aplicación. He diseñado tres paneles distintos porque cada usuario necesita cosas diferentes.

5.1. Panel del Alumno



- **Contador de Horas**: Una barra de progreso que te dice de un vistazo cuánto te queda para terminar.
- **Seguimiento Semanal**: Un espacio para escribir qué has hecho cada día. Esto elimina los informes pesados al final del mes.
- **Botón de Incidencias**: Si hay un marrón (tareas que no tocan, problemas de horario), se reporta aquí oficialmente.
- **Chat**: Hablar con el tutor de forma directa y profesional.

5.2. Panel del Tutor

- **Lista de Alumnos**: El tutor puede saltar de un alumno a otro para ver cómo van.
- **Alertas**: Indicadores visuales (como puntos rojos) que avisan si alguien ha reportado una incidencia o no ha rellenado el diario.

5.3. Panel del Centro Educativo

- **Gestión de Tutores**: Ver cuánta carga de trabajo tiene cada profesor.
- **Estadísticas**: Un resumen de cuántos alumnos hay en prácticas y qué convenios están activos.

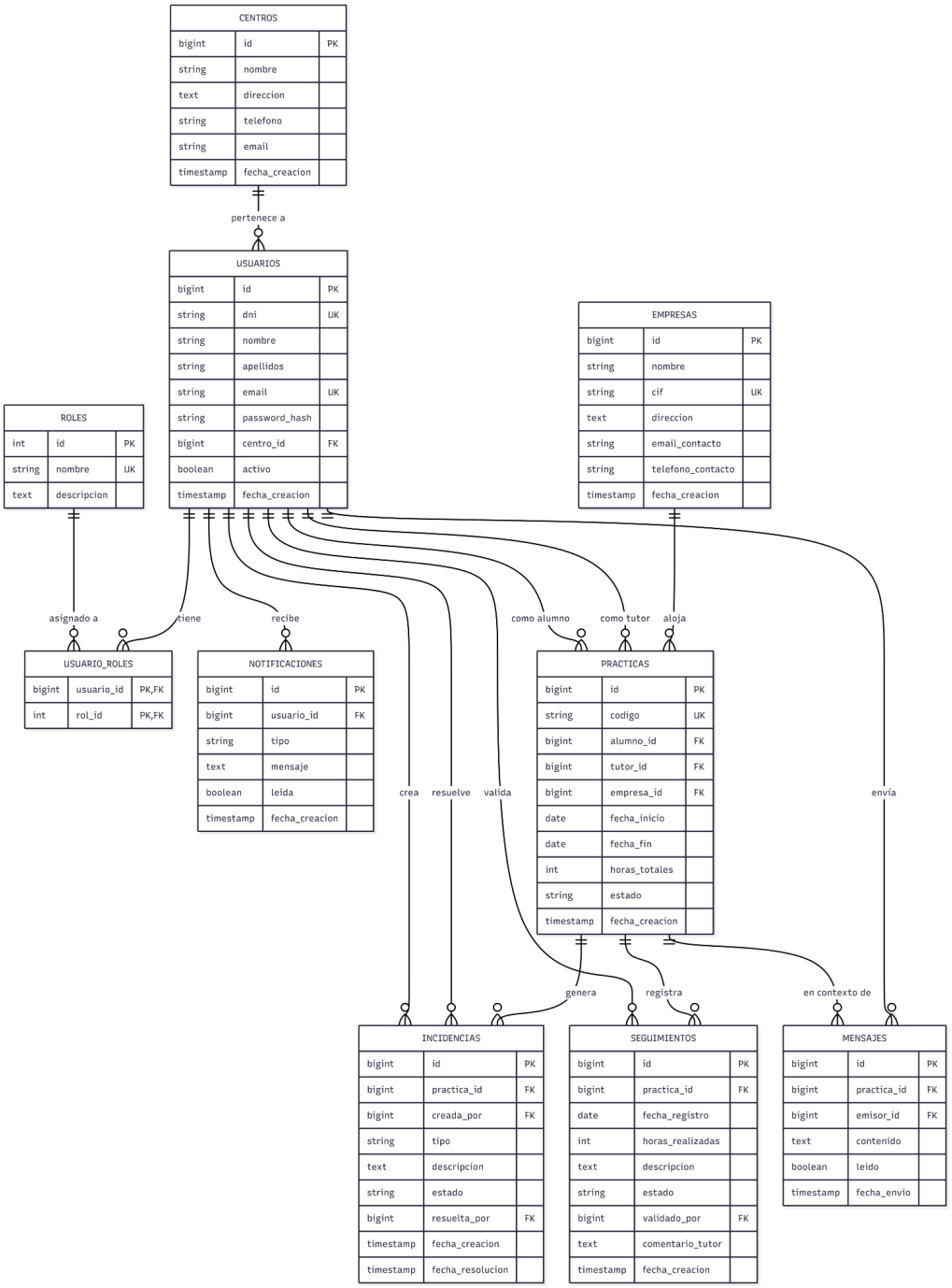
6. Modelo de la Base de Datos

Para que todo esto funcione por detrás, he diseñado un sistema de tablas conectadas:

- **Usuarios:** Donde guardamos quién es alumno, quién es tutor y quién es del centro.
- **Prácticas:** La tabla principal que une al alumno con su empresa y su tutor.
- **Seguimientos e Incidencias:** Registros de lo que se va haciendo y de los problemas que surgen.
- **Chat:** Guardamos todos los mensajes para que haya un historial de lo hablado.

Durante el desarrollo, detecté que un alumno de FP necesita dos responsables: el **Tutor del Centro** y el **Tutor de la Empresa**. He actualizado el modelo para separar ambas figuras en la tabla de prácticas, garantizando un seguimiento académico y laboral independiente..

El resultado final de las tablas interconectadas en PostgreSQL es el siguiente diagrama de Entidad-Relación:



7. Planificación (Cómo lo voy a hacer)

He dividido el trabajo en 16 semanas para llegar bien a la entrega:

1. **Semanas 1-3:** Analizar todo bien y diseñar las tablas de la base de datos.
2. **Semanas 4-7:** Programar el cerebro (backend) y el sistema para entrar en la web (login).
3. **Semanas 8-12:** Montar las pantallas (frontend), el chat y el sistema de incidencias.
4. **Semanas 13-16:** Probar que todo funcione, corregir fallos y terminar de escribir esta memoria.

Estado Actual (Entrega Hito 1 - 25%): Actualmente, he completado las Semanas 1 a 3 (Análisis y diseño de la base de datos con Flyway en PostgreSQL) y he avanzado drásticamente en las Semanas 4 a 7. El cerebro (Backend en Spring Boot) ya tiene configurada la estructura base, el sistema de cifrado de contraseñas, la generación de tokens JWT y los endpoints funcionales para el Login, el Registro y la consulta de entidades maestras (Centros y Empresas).

8. Conclusión y Futuro

Este proyecto demuestra que se puede mejorar mucho la experiencia de las prácticas si dejamos de usar el correo para todo. Al centrarme en el seguimiento, consigo una herramienta que realmente ayuda al alumno y al tutor en su día a día.

En el futuro, la idea es que la aplicación también gestione la firma digital de los convenios de las empresas y que envíe notificaciones al móvil para que nadie se olvide de rellenar sus horas y el uso de IA para analizar el grado de satisfacción del alumno a través de sus diarios de prácticas. Vamos, encargarse también no solo de la gestión de cuando el alumno ya esta de practicas, si no también encargarse de todo el proceso para conseguir prácticas que la empresa suba cuantas personas y los perfiles que necesita, el alumno preferencias, y que se le da mejor, y el centro se encargue de hacer ese enlace entre empresa y alumno. Pero eso es plan de futuro.